

```

local lookup = local b64chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ-
abcdefghijklmnopqrstuvwxyz0123456789+/' for i = 1, #b64chars do lookup[b64chars:sub(i,i)]
= i-1 end

```

```

local function b64decode(s) s = s:gsub('[^A-Za-z0-9+/=]', '') local out =
local i = 1 while i <= #s do local a = lookup[s:sub(i,i)] or 0 local b = lookup[s:
sub(i+1,i+1)] or 0 local c = s:sub(i+2,i+2) local d = s:sub(i+3,i+
3) local cv = (c == '=') and 0 or (lookup[c] or 0) local dv = (d == '=') and 0 or (lookup[d] or 0) local n =
a*262144+b*4096+cv*64+dv out[out+1] = string.char(math.floor(n/65536)) if c == '='
then out[out+1] = string.char(math.floor(n/256)) if d == '=' then out[out+
1] = string.char(n%256) i = i+4 end return table.concat(out) end

```

```

local b64data = [[iVBORw0KGgoAAAANSUhEUgAAAOsAAADKCAMAAAD6r/rFAAAABGdBT

```

```

local f = io.open('nit_logo.png', 'wb') if f then f:write(b64decode(b64data)); f:
close() end

```



NUMIDIA INSTITUTE OF TECHNOLOGY

Department of Computer Science

Web Development Project Report • Academic Year 2024–2025

Abstract

The **Internet Artifact Archive** is a full-stack web application built to catalog, preserve, and collectively investigate forgotten pieces of internet history. Users file structured case entries covering lost media, defunct websites, unexplained viral phenomena, and alternate reality game fragments. The platform wraps each entry in a living investigation layer: community discussion threads, peer-reviewed long-form analyses, chronological event timelines, and a voting system that surfaces the most substantiated findings to the top.

Authentication, role-based access control, content moderation, and all business logic are enforced at the database layer through PostgreSQL row-level security policies and server-side triggers, making the security model entirely independent of the client application.

This report covers the full scope of the project: the rationale for the chosen domain, the complete technical architecture, a feature-by-feature walkthrough of every implemented requirement, the database schema with its entities and relationships, the trigger-based business logic, the security model, and the user experience design decisions.

Keywords: web application, React, Supabase, PostgreSQL, row-level security, community platform, digital preservation, internet archive.

Contents

1 Introduction

1.1 Domain Selection and Rationale

The project brief specified a *Service and Product Exchange Platform*: a system connecting people who request things with people who provide them. At its core, that structure is a publishing and matchmaking problem. A user posts something; others respond; a conversation accumulates over time. The Internet Artifact Archive applies that exact structure to a domain with a genuine need for it: the preservation and community investigation of forgotten internet history.

The internet generates culture faster than any institution can document it. Content disappears without warning. Platforms shut down overnight, taking years of community-generated material with them. Websites go dark, media files become inaccessible, and entire cultural phenomena persist only in partial mirrors and fragmented memory. The Internet Artifact Archive is a structured answer to that problem.

The domain maps cleanly onto the brief. A user “files a case” by submitting an artifact: a piece of lost or unexplained content brought to the community’s attention. Other users respond by contributing analyses, adding timeline entries, and voting on the quality of the investigation. The artifact’s status advances as understanding grows. The “request” is the open question surrounding the artifact; the “service” is the collective investigation the community provides.

Choosing this domain over a generic exchange platform was deliberate. Every feature required by the brief gains a specific, motivated purpose within an archive context. Status tracking becomes a research progress indicator. Filtering by category helps researchers narrow a vast catalog. The voting system prioritizes verified or well-sourced findings above speculation. The platform is not a demonstration of web application features; it uses those features to solve a real problem.

1.2 Project Scope

The Internet Artifact Archive implements all ten requirements of the project brief and extends them with community features that make the application genuinely functional as a preservation platform. The standard requirements are fully covered: user account management with distinct roles, authenticated and authorized access, full CRUD on user-submitted entries, structured data tables with sorting and pagination, validated forms with inline error handling, keyword and categorical search, a dynamic filtering system, a personal dashboard, and an administrator panel with moderation tools.

Beyond the brief, the platform adds community voting with server-enforced score calculation, a peer-reviewed analysis system, chronological event timelines per artifact, user-curated collections, a real-time community message board with anonymous

participation support, and an automatic badge system driven entirely by database triggers.

2 System Architecture

2.1 Technology Stack

The Internet Artifact Archive is built on a two-layer architecture: a React single-page application on the frontend and Supabase as the complete backend infrastructure. There is no Node.js server, no Express middleware layer, and no custom REST API. The database communicates directly with the browser through PostgREST, an automatic REST interface that Supabase generates from the PostgreSQL schema. Access control is governed entirely by row-level security policies defined within the database itself.

Layer	Technology	Version
Frontend	React	18.3.1
Build tool	Vite	5.4.10
Client-side routing	React Router DOM	6.30.0
Animation library	GSAP (GreenSock)	3.15.0
Backend database	Supabase (PostgreSQL)	Latest
Authentication	Supabase Auth	Latest
File storage	Supabase Storage	Latest
Deployment	Netlify	—

Table 1: Full technology stack

The decision to use Supabase as the sole backend eliminates an entire infrastructure tier. Authentication, file uploads, real-time subscriptions, and database access are all handled through a single SDK (`@supabase/supabase-js` v2.49.4). The frontend focuses entirely on user experience, while the database handles every access rule, rate limit, score computation, and badge award. No application server can be compromised because no application server exists.

The Supabase anonymous key is intentionally public; its permissions are defined by RLS policies, not by keeping it secret. Every operation the client performs runs under the authenticated user’s identity inside PostgreSQL, where the policies evaluate it.

2.2 Project Structure

The repository is organized around three top-level concerns: the React application under `src/`, the static HTML entry point, and the complete database definition under `supabase/`.

```

echo_chamber/
+-- src/
|   +-- main.jsx           React entry point
|   +-- App.jsx            Root component, routing, global auth
|   state
|   +-- styles.css         Global stylesheet (vintage archive
|   aesthetic)
|   +-- lib/
|   |   +-- supabase.js     Supabase client singleton
|   |   +-- badges.js       Badge definitions and lookup utilities
|   |   +-- errorUtils.js   Error formatting and safe async wrappers
|   +-- components/
|   |   +-- AppShell.jsx     Header, navigation bar, user menu
|   |   dropdown
|   |   +-- BadgeDisplay.jsx Badge chip renderer with overflow
|   |   handling
|   |   +-- CardSwap.jsx     GSAP 3D animated card deck component
|   |   +-- ErrorBoundary.jsx React render error fallback wrapper
|   +-- pages/
|   |   +-- HomePage.jsx     Landing page, community wall, featured
|   |   artifacts
|   |   +-- CatalogPage.jsx  Browse, search, filter, sort, paginate
|   |   +-- ArtifactPage.jsx Full artifact detail, tabs, voting,
|   |   reports
|   |   +-- PostPage.jsx     Submit new artifact with image upload
|   |   +-- DashboardPage.jsx Personal user area: stats, artifacts,
|   |   collections
|   |   +-- AdminPage.jsx    Administration panel, charts, user
|   |   management
|   |   +-- AuthPage.jsx     Login, registration, password reset
|   |   +-- ProfilePage.jsx  Account settings, avatar, password
|   |   change
|   |   +-- UpdatePasswordPage.jsx Password reset via email link
|   |   +-- AboutPage.jsx    Platform mission and usage guide
|   |   +-- NotFoundPage.jsx 404 fallback page
+-- supabase/
|   +-- schema.sql          Full schema: tables, RLS, triggers,
|   views
|   +-- seed_mock_posts.sql Optional demo data for development
|   +-- verify.sql          Database verification helpers
+-- public/
+-- index.html
+-- vite.config.js
+-- package.json

```

Listing 1: Repository directory layout

2.3 Architectural Principles

Database as the single authority.

Every business rule lives in PostgreSQL. Rate limits are before-insert triggers. Access restrictions are RLS policies. Score calculations are after-vote triggers. A user who disables JavaScript, inspects and replays network requests, or crafts raw PostgREST calls cannot circumvent any of these rules, because they do not exist in JavaScript.

PostgREST as the data interface.

Supabase exposes the PostgreSQL schema as a REST API automatically. The client uses `.select()`, `.insert()`, `.update()`, and `.delete()` method chains on table and view names, which PostgREST translates into parameterized SQL and executes under the calling user's authenticated identity.

Views for flattened reads.

Two PostgreSQL views, `artifacts_feed` and `wall_posts_with_cheers`, flatten the most frequent read patterns into single queries. The catalog page and community wall each query a single view rather than issuing multi-table joins from application code.

Serverless, stateless frontend.

The React application holds no server-side state. All persistent data lives in Supabase. The application can be served from any static host and scales horizontally without coordination.

3 User Account Management

3.1 Registration

New users register with three fields: an email address, a password, and a username. The application calls `supabase.auth.signUp()` with the email and password, passing the username as metadata. Upon successful registration, a trigger on `auth.users` fires automatically: it inserts a corresponding row into the `profiles` table and awards the new user their first badge, "Case Opened." If the requested username conflicts with an existing one, the trigger appends a short random numeric suffix to guarantee uniqueness without rejecting the registration.

All three fields are required. The password input is accompanied by a confirmation field; the client validates that both values match before submitting to Supabase. Email format validation and duplicate address rejection are handled by Supabase Auth. Username uniqueness is enforced by a `UNIQUE` constraint on the `profiles` table.

3.2 Login and Logout

Login accepts an email address and a password, submitted to `supabase.auth.signInWithPassword()`. A successful call returns a session object containing a JWT and a refresh token, both stored in the browser's `localStorage` and managed automatically by the SDK. Tokens are refreshed before expiry without any application-level handling.

On every page load, `App.jsx` calls `supabase.auth.getSession()` to restore an existing session, providing a persistent login experience across browser tabs and sessions. The component also subscribes to `supabase.auth.onAuthStateChange()`, which fires whenever a session is created, refreshed, or destroyed. The handler fetches the user's profile from the `profiles` table and assembles a combined user object (session data plus profile fields) passed as a prop through the component tree.

A 12-second timeout is applied to both the session restoration call and the subsequent profile fetch. If either exceeds the limit, the application enters a limited browsing mode, granting read access to the public catalog without blocking the entire page on a slow connection.

Logout calls `supabase.auth.signOut()`, which clears the stored tokens and triggers the auth state listener. The UI updates immediately.

3.3 Profile Management

Each user has a dedicated profile page at `/profile`. From there, users can update their username (2 to 50 characters) and biography (up to 500 characters). Avatar support is included: the user selects an image file (JPEG or PNG, maximum 2 MB), which is uploaded to Supabase Storage under a path scoped to their user ID. The resulting public URL is written to the `avatar_url` column on their profile and displayed in the navigation header.

Profile updates are RLS-restricted: the `UPDATE` policy checks `auth.uid() = id`, ensuring a user can only modify their own row. Administrators bypass this restriction and may edit any profile row.

3.4 Password Management

Two distinct flows handle password changes. Authenticated users update their password directly from the profile page: they enter a new value twice, the client confirms the values match, and the application calls `supabase.auth.updateUser({password})`. No current-password verification step is required, as a live session already proves identity.

For forgotten passwords, the login page offers a reset option. The user enters their email address; the application calls `supabase.auth.resetPasswordForEmail(email, {redirectTo})` with a redirect target of `/auth/update-password`. Supabase sends a password reset email containing a time-limited link. When the user follows the link, `UpdatePasswordPage.jsx` validates the recovery session before rendering the new password form, preventing the page from being accessed without a valid reset token. A

60-second cooldown applies to the reset email request to limit abuse.

3.5 Account Deletion

Users can delete their own account from the profile page. Deletion cascades through the database via foreign key constraints: all artifacts, analyses, comments, collections, and votes belonging to the user are removed automatically. The `auth.users` record is deleted through the Supabase Admin API, permanently invalidating the credentials.

3.6 User Roles

The platform defines three effective permission levels.

Unauthenticated visitor Can browse the full catalog, read artifact details, and view the community message board. Cannot create, edit, vote, comment, or report.

Registered user Can submit artifacts, write analyses and comments, add timeline entries, vote on artifacts and analyses, create and manage collections, report content, and post to the community wall. A ban flag on the profile suspends all write access while preserving read access.

Administrator Has unrestricted read and write access to all records. Can edit or delete any artifact regardless of ownership, manage user accounts (ban, unban, manually assign badges), review and resolve content reports, and manage the category list through the admin panel.

The `is_admin` boolean on the `profiles` table controls administrator privileges. A database trigger prevents any non-administrator from setting this flag to `true` on their own or any other profile, even through a direct API call with a crafted request body.

4 Authentication and Authorization

4.1 Session Management

Supabase Auth manages the full session lifecycle. The JWT issued on sign-in contains the user's `auth.uid()`, which PostgreSQL's RLS system reads on every query to evaluate access policies. The SDK handles token refresh transparently; no custom session renewal logic is needed in the application.

The root `App.jsx` component holds the global authentication state. It restores any persisted session on mount, subscribes to state changes for the lifetime of the application, and provides the assembled user object to every page through props. Pages that need user data read from this prop rather than making their own auth calls.

4.2 Role-Based Access Control

Access control operates at two levels.

Routing layer. Two React wrapper components gate page access. `ProtectedRoute` checks for a non-null user object and redirects unauthenticated visitors to `/auth`. `AdminRoute` extends `ProtectedRoute` and additionally checks the `is_admin` flag; any authenticated non-administrator is redirected to the home page.

Database layer. RLS policies on every table define exactly what each authenticated identity may read, insert, update, and delete. These policies evaluate inside PostgreSQL and cannot be bypassed by the client. A representative selection is shown below.

```

1  -- Public read access on all artifacts
2  CREATE POLICY "artifacts_select_all"
3    ON artifacts FOR SELECT USING (true);
4
5  -- Only the owner or an admin may update an artifact
6  CREATE POLICY "artifacts_update_owner_admin"
7    ON artifacts FOR UPDATE USING (
8    auth.uid() = user_id
9    OR EXISTS (
10     SELECT 1 FROM profiles
11     WHERE id = auth.uid() AND is_admin = true
12   )
13 );
14
15 -- Banned users cannot insert new artifacts
16 CREATE POLICY "artifacts_insert_authenticated"
17   ON artifacts FOR INSERT WITH CHECK (
18   auth.uid() = user_id
19   AND NOT EXISTS (
20     SELECT 1 FROM profiles
21     WHERE id = auth.uid() AND is_banned = true
22   )
23 );
24
25 -- Only admins can read or update content reports
26 CREATE POLICY "reports_admin_only"
27   ON reports FOR SELECT USING (
28   EXISTS (
29     SELECT 1 FROM profiles
30     WHERE id = auth.uid() AND is_admin = true
31   )
32 );

```

Listing 2: Representative row-level security policy definitions

4.3 Protected Routes

The following pages require authentication and redirect to `/auth` when accessed without a session.

- `/post` — submit a new artifact
- `/dashboard` — personal user area
- `/profile` — account settings and password management

The `/admin` page requires both a valid session and the administrator flag. All other pages are publicly accessible, though certain interactions on those pages (voting, commenting, cheering) require a session and prompt unauthenticated users to sign in.

5 Artifact Management

Within the Internet Artifact Archive, the “entries” described by the project brief take the form of **artifacts**: documented pieces of internet history submitted for community investigation. Each artifact functions simultaneously as a catalog entry and a living case file. The relationship between the brief’s requirements and the platform’s implementation is direct: submitting an artifact corresponds to posting a request or offer; analyses and comments correspond to provider responses; the status field tracks the entry through its lifecycle.

5.1 Submitting Artifacts

The submission form at `/post` collects the fields shown in Table ??.

Field	Input type	Constraints
Title	Text input	Required, 1–300 characters
Description	Textarea	Required, 1–5000 characters
Category	Select + create	Required; new categories created in-line
Artifact date	Date picker	Required
Initial status	Select	One of four defined status values
Source links	Dynamic URL list	Optional; multiple URLs supported
Images	File upload	Optional; up to 30 files, 5 MB each

Table 2: Artifact submission form fields

Source links are managed through a dynamic form: clicking “Add link” appends a new URL input; clicking the remove button beside any entry deletes it. The resulting list is serialized as a JSON array and stored in the `source_link` text column.

Image uploads follow a similar pattern. Each selected file is uploaded individually

to Supabase Storage under the path `{user_id}/{timestamp}/{index}-{filename}`. After all uploads complete, the list of public URLs is serialized as a JSON array and stored in `media_url`. Preview thumbnails render next to each selected file before submission, allowing users to review and remove images before they are uploaded.

Category selection includes a live “create new” option: if the user types a name not present in the existing list, the submission creates the category in the `categories` table first, then assigns the new category to the artifact. This keeps the category taxonomy organic and community-driven.

A database-level rate limit prevents abuse: no user may submit more than five artifacts per hour. This limit is enforced by a before-insert trigger and cannot be circumvented from the client.

5.2 Editing Entries

Owners and administrators can edit any field on an existing artifact from its detail page. The form pre-populates with current values. Images and source links can be added or removed individually without replacing the full set. Changes are saved with an `.update()` call restricted by an RLS `UPDATE` policy; the `updated_at` timestamp is refreshed automatically by a database trigger.

5.3 Deleting Entries

Deletion is available to the artifact’s owner and to administrators. Selecting the delete action from the detail page triggers a confirmation step before committing. The delete operation cascades through the database: all associated analyses, comments, timeline entries, votes, and collection membership records are removed by foreign key constraints with `ON DELETE CASCADE`.

5.4 Viewing Details

The artifact detail page at `/artifact/:id` presents the full case file. The upper section displays title, category, origin date, current status, author username, and community vote score. Below that, a media gallery renders any attached images in a grid layout. A source links section lists all submitted URLs as labeled, clickable anchors.

The lower half of the page is divided into three tabs.

Discussion The comment thread, ordered chronologically. Registered, non-banned users may add comments up to 1000 characters. A database-level rate limit caps submissions at 20 comments per user per hour.

Timeline A chronological log of significant events related to the artifact. Any registered user may add entries up to 2000 characters. The timeline builds a narrative history: when a file was first reported missing, when a partial recovery was made, when a researcher identified a creator.

Analysis Long-form peer-reviewed writeups, each up to 5000 characters, ranked by community vote score. Investigators contribute their most thorough findings; the community votes the most substantiated analyses to the top. This is the platform’s primary knowledge-building mechanism.

5.5 Status Lifecycle

Each artifact carries a status field reflecting the current state of investigation. Table ?? defines the four values.

Status	Meaning
Unknown	Newly submitted; origin, authorship, or nature not yet determined
Under Investigation	The community is actively researching the artifact
Explained	Origin, meaning, or authorship has been established
Archived	Investigation closed; entry preserved for the historical record

Table 3: Artifact status values and their meanings

The **status** column is constrained by a **CHECK** clause to these four values. Owners and administrators may update the status at any time from the detail page.

6 Data Visualization with Tables

6.1 Catalog Browse View

The catalog page at `/catalog` presents artifacts in a structured feed styled after community aggregator sites. Each row consists of a narrow scoring column on the left (vote score with upvote and downvote controls) and a content column on the right (title, category badge, description excerpt, submission date, and author). Action buttons appear inline on rows the current user owns, giving immediate access to edit and delete without navigating away.

6.2 Dashboard Tables

The personal dashboard presents three structured content lists.

- **My Artifacts:** All artifacts the user has submitted, with title, status badge, and submission date. Each row links to the full detail view.
- **My Contributions:** All analyses the user has written, each linked back to its parent artifact by title.

- **My Collections:** All collections the user owns, with item count and inline management controls.

6.3 Administration Tables

The admin panel includes two tabular datasets. A contributor leaderboard ranks all users by total submission count. A user management table lists every registered account with columns for username, email address, join date, administrator flag, ban status, and action buttons for moderation operations.

6.4 Sorting, Pagination, and Action Buttons

The catalog supports four sort orders: newest first, oldest first, highest vote score, and alphabetical by title. Pagination divides the result set into pages of five entries. The current page number, total count, and previous/next navigation controls are displayed below the feed. All three features compose cleanly: changing any one resets to page one and re-runs the query with all active constraints applied simultaneously.

7 Forms and Data Entry

The platform includes eight distinct forms. All use React controlled components; all validate required fields client-side before making any network call; all surface error messages inline rather than through alerts or page reloads.

7.1 Registration and Login Forms

The registration form collects email, username, and password with confirmation. The login form collects email and password. A “Forgot password?” link on the login form switches to the password reset view without navigation. Both forms display field-level error messages returned by Supabase Auth, translated through `lib/errorUtils.js` into plain-language descriptions.

7.2 Artifact Submission Form

Seven distinct input types in a single form, described in detail in Section ?? (Table ??). Notable behaviors: the category field acts as a searchable dropdown with inline creation; the image upload previews files before committing the upload; the source link list grows and shrinks dynamically without re-rendering the entire form.

7.3 Profile Editing Form

Separate controls for username, biography, and avatar image. The avatar input accepts image files only and enforces a 2 MB client-side size check before initiating the Supabase Storage upload. All three fields are optional on any given save; submitting with an empty avatar field does not overwrite an existing one.

7.4 Analysis and Comment Forms

Inline textarea forms on the artifact detail page. Analyses allow up to 5000 characters; comments up to 1000. Both display live character counts and enforce their limits before the Supabase call is made, giving the user immediate feedback before any network activity occurs.

7.5 Collection Forms

A two-field form (name, optional description) for creating new collections on the dashboard. A separate inline search-and-add form allows adding artifacts to a selected collection by searching for its title.

7.6 Validation Approach

All required fields are checked before the Supabase call fires. Server responses are caught in `try/catch` blocks and passed through `lib/errorUtils.js`, which maps PostgreSQL error codes, PostgREST status strings, check constraint violations, and rate limit exceptions to short, specific user-facing messages. A 23505 unique violation becomes “That username is already taken.” A rate limit trigger exception becomes “You are submitting too quickly. Please wait before trying again.” Users never see raw error objects.

8 Search Functionality

The catalog page includes a keyword search input. On submission, the search term filters the artifact feed against three columns: `title`, `description`, and the submitting user’s `username` (available through the `artifacts_feed` view). All three use case-insensitive pattern matching.

```
1 SELECT *
2 FROM artifacts_feed
3 WHERE title      ILIKE '%keyword%'
4      OR description ILIKE '%keyword%'
5      OR username  ILIKE '%keyword%'
6 ORDER BY created_at DESC
7 LIMIT 5 OFFSET 0;
```

Listing 3: Keyword search applied across the `artifacts_feed` view

Search composes with all active filters: a keyword search while a category filter is applied returns only artifacts matching both constraints simultaneously. Clearing the search field while filters remain active re-runs the query with filters intact.

Two supplementary navigation aids appear in the catalog sidebar. A **tag cloud** offers single-click filtering on common topic keywords (“lost media,” “ARG,” “internet culture,”

“mystery,” and others), each functioning as a pre-filled keyword search. An **archive timeline** lists all years for which at least one artifact has been submitted, alongside entry counts per year. Clicking a year populates the date-range filter with that calendar year’s bounds, immediately narrowing the catalog to artifacts from that period.

9 Filtering System

The catalog filter panel sits above the artifact feed and exposes four independent filters. All filters combine through AND logic, and all update the feed without a full page reload.

Category filter A dropdown populated from the live `categories` table. Selecting a value appends `.eq('category_id', id)` to the Supabase query.

Status filter A dropdown presenting the four artifact statuses. Selecting a value appends `.eq('status', value)`.

Date range filter Two date inputs (from and to). Each active bound appends a `.gte()` or `.lte()` constraint on `artifact_date`. Either bound may be set independently.

Sort order A dropdown switching between newest, oldest, most-voted, and alphabetical by title. The active sort is applied as an `.order(field, {ascending})` call.

All filter state lives in React local state within `CatalogPage.jsx`. A single `fetchArtifacts` function reads the current state of all active filters before building the Supabase query, so each filter input is decoupled from the others and the query construction logic is centralized in one place. Resetting any filter immediately re-runs the query without it; changing a filter resets pagination to page one.

10 Dashboard

The personal dashboard at `/dashboard` is the user’s operational home on the platform. It opens with a statistics summary card presenting three numbers: total artifacts filed, total analyses submitted, and total upvotes received across all artifacts. These figures are computed from live Supabase queries on each page load so they always reflect the current state.

My Artifacts lists every artifact the user has submitted, showing title, status as a color-coded badge, and submission date. Each row links directly to the artifact’s detail page.

My Contributions lists every analysis the user has written, paired with the title of its parent artifact as a link. This allows contributors to track every investigation they have participated in from a single view.

My Collections lists every collection the user owns, with item count. Each collection expands to show its current contents with per-item removal buttons. Above the list, a form allows the user to search for any artifact by title and add it to a selected collection. A second form below creates a new collection with a name and optional description. Both forms submit and update the list without navigating away from the dashboard.

11 Administration Panel

The admin panel at `/admin` is accessible only to users with `is_admin = true` in their profile. Attempting to access it as a regular user redirects to the home page. The panel is organized into four operational sections.

11.1 Activity Charts

The top of the panel displays a 14-day activity overview in bar chart form. Four metrics are charted independently: artifacts submitted per day, analyses written per day, comments posted per day, and new users registered per day. Data is retrieved by querying each respective table for records with `created_at` within the past 14 days, then grouped into daily buckets client-side and rendered as inline SVG bar charts. Administrators can identify unusual spikes in activity or submission patterns at a glance.

11.2 User Management

A full table of all registered users presents username, email address, join date, administrator status, and ban status for each account. Each row includes action buttons. Banning a user sets `is_banned = true` on their profile, which the RLS INSERT policies across all content tables evaluate before permitting writes. A badge management form on each user row allows administrators to assign or revoke individual badges manually, covering cases such as verified researchers or community veterans who have not yet earned a badge through the automated trigger system.

11.3 Moderation Queue

Any registered user can report an artifact from its detail page by clicking the “Report” button and entering a reason. All open reports appear in the moderation queue on the admin panel. Each entry displays the reported artifact’s title (linked), the reporting user’s username, the stated reason, and the submission timestamp. Administrators resolve reports by clicking the “Resolve” button, which updates the status to `'Resolved'` and removes the entry from the active queue. The `reports` table has RLS policies

permitting only administrators to read or modify it; regular users can insert reports but cannot read or edit them.

11.4 Category Management

Administrators can add new categories through an inline form. Existing categories are listed with edit and delete controls. Deleting a category is blocked by the foreign key constraint on the `artifacts` table if any artifacts currently reference it, preventing orphaned records.

12 Extended Platform Features

Beyond the project brief requirements, the Internet Artifact Archive implements several features that transform it from a catalog application into a functioning research community.

12.1 Voting and Scoring

Every artifact and every analysis can be upvoted (+1) or downvoted (-1) by any registered, non-banned user. Vote scores are stored as integer columns directly on the `artifacts` and `analyses` tables. A pair of database triggers recompute each score from the vote table after every insert, update, or delete operation, ensuring the stored score is always the authoritative sum regardless of concurrent activity.

Self-voting on artifacts is blocked by an RLS INSERT policy. The `artifact_votes` table caches the artifact owner's user ID in an `artifact_owner_id` column (maintained by a separate trigger that copies it from the `artifacts` table on insert). The INSERT policy checks `auth.uid() <> artifact_owner_id` and rejects self-votes at the policy level.

Votes are submitted with an upsert operation using `onConflict: 'artifact_id,user_id'`. This allows a user to change an existing vote (from +1 to -1 or vice versa) by replacing the prior record rather than creating a duplicate. Removing a vote deletes the row outright and triggers a score recomputation.

12.2 Badge System

Six badges are awarded automatically based on contribution history and account age. The full award conditions are listed in Table ??.

Badges are stored as a text array in the `profiles.badges` column. A PostgreSQL function `refresh_badges(user_id)` evaluates all conditions for a given user by querying current counts and the account creation date, then writes the resulting array back to the profile. Database triggers on the `artifacts`, `analyses`, and `collections` tables call this function after each relevant insert or delete. Badge state is therefore always

Badge	Trigger event	Award condition
Case Opened	Account registration	Always awarded on signup
Contributor	Artifact insert / delete	≥ 3 artifacts filed
Archivist	Analysis insert / delete	≥ 3 analyses submitted
Investigator	Analysis insert / delete	≥ 2 analyses on Explained artifacts
Curator	Collection insert / delete	≥ 1 collection created
Veteran	Any of the above events	Account age ≥ 30 days

Table 4: Badge award conditions and trigger events

current; no background job or scheduled task is needed.

The `BadgeDisplay` component renders each badge as a color-coded chip. A `max` prop limits how many chips are shown inline; any overflow is summarized as “+N more,” keeping the navigation header and profile display uncluttered.

12.3 Community Wall

The home page includes a real-time community message board. Any visitor may post a message of up to 280 characters; registered users post under their username, while anonymous visitors are assigned a generated alias stored in `sessionStorage`. The alias format (`anonymous[0-9]{3,}(-[a-z]+-[a-z]+)?`) is enforced by a `CHECK` constraint in the database, preventing arbitrary strings from being submitted as display names.

The wall shows posts from the past 24 hours, capped at 180 entries per view. Registered users can react to any post with a “cheer” (a lightweight acknowledgment reaction), and posts may reply to a parent post, forming shallow two-level threads. The wall refreshes every 15 seconds to surface new content.

The `wall_posts_with_cheers` view aggregates cheer counts per post and computes a per-user `cheered_by_user` boolean using `auth.uid()` inside the view definition, so the interface renders each post’s reaction state without a separate join or query.

12.4 Collections

Registered users can create named collections grouping any set of artifacts. Each collection has a name (required, 1 to 200 characters) and an optional description (up to 500 characters). Collections are publicly visible and linked from the creator’s dashboard. Only the owner may modify a collection’s contents; this restriction is enforced by RLS policies on both the `collections` and `collection_items` tables. The `collection_items` table uses a composite primary key (`collection_id`, `artifact_id`), preventing duplicate entries.

12.5 Content Reporting

The artifact detail page exposes a “Report” button to registered users. Clicking it opens a form with a free-text reason field. Submitted reports are inserted into the `reports` table with a default status of `'Open'`. Only administrators can read the reports table; regular users can submit reports but have no visibility into others'. The moderation workflow is described in Section ??.

13 Database Design

13.1 Schema Overview

The database consists of eleven tables, two views, approximately fourteen triggers, and around forty row-level security policies. The full schema is defined in a single idempotent SQL file (`supabase/schema.sql`) using `CREATE ... IF NOT EXISTS` and `DROP ... IF EXISTS` patterns so it can be applied cleanly to a fresh database or re-applied after changes without errors. The file is approximately 800 lines.

13.2 Core Entity Tables

`profiles` extends Supabase’s built-in `auth.users` table with application-level data. The `id` column is a foreign key referencing `auth.users(id)` with `ON DELETE CASCADE`. Username is constrained to 2–50 characters with a `UNIQUE` index. The `badges` column is a `text[]` array. Booleans `is_admin` and `is_banned` default to `false`.

`artifacts` is the central entity of the application. Key columns: `user_id` (owner, foreign key to profiles), `title` (1–300 chars), `description` (1–5000 chars), `category_id` (foreign key to categories), `source_link` (JSON array serialized as text), `media_url` (JSON array of Storage URLs serialized as text), `artifact_date` (date), `status` (check-constrained text), `vote_score` (integer, trigger-maintained). Indexes exist on `category_id`, `status`, `artifact_date`, and `vote_score DESC` to support catalog filter and sort operations efficiently.

`categories` is a simple lookup table: auto-increment `id` and a `UNIQUE name`. Seeded with six initial values: Lost Media, Old Websites, Internet Culture, ARGs, Digital Art, and Forums/Communities.

`analyses` and `comments` each carry foreign keys to both `artifact_id` and `user_id`. Analyses additionally have a trigger-maintained `vote_score` integer. `timeline_entries` follow the same two-key pattern without voting.

`artifact_votes` is keyed on the composite primary key (`artifact_id`, `user_id`). It additionally caches `artifact_owner_id` for use by the self-vote prevention RLS policy. `analysis_votes` is keyed on (`analysis_id`, `user_id`).

collections holds user-owned named groups. **collection_items** links collections to artifacts through a composite primary key (`collection_id`, `artifact_id`).

wall_posts has a nullable `user_id` to support anonymous posts. The `alias` display name column is constrained by the anonymous format check. A self-referencing `reply_to` foreign key supports thread replies.

wall_post_cheers is keyed on (`post_id`, `user_id`).

reports references `artifact_id` and `user_id`, with a `status` column constrained to 'Open' or 'Resolved', defaulting to 'Open'.

13.3 Views

artifacts_feed joins artifacts, categories, and profiles into a flat view. Denormalizing `category_name` and `username` at the view level means the catalog page never issues three-table joins from application code; it queries a single view with filters applied.

wall_posts_with_cheers aggregates cheer counts per post through a `LEFT JOIN` on `wall_post_cheers` and computes the current user's cheer status as a subquery boolean using `auth.uid()` inside the view definition. Explicit `GRANT SELECT` statements allow the view to be queried under the public role.

13.4 Triggers and Business Logic

Rate limiting is the most security-critical trigger group. Before-insert triggers on artifacts, analyses, comments, and votes count the calling user's submissions in the past 60 minutes and raise an exception if the respective limit is exceeded. The limits are: 5 artifacts/hour, 10 analyses/hour, 20 comments/hour, and 50 votes/hour combined across artifact and analysis votes.

```

1 CREATE OR REPLACE FUNCTION check_artifact_rate_limit()
2 RETURNS trigger AS $$
3 BEGIN
4     IF (
5         SELECT COUNT(*) FROM artifacts
6         WHERE user_id = NEW.user_id
7             AND created_at > NOW() - INTERVAL '1 hour'
8     ) >= 5 THEN
9         RAISE EXCEPTION
10            'Rate limit exceeded: max 5 artifacts per hour';
11     END IF;
12     RETURN NEW;
13 END;
14 $$ LANGUAGE plpgsql SECURITY DEFINER;
15
16 CREATE TRIGGER artifact_rate_limit
17 BEFORE INSERT ON artifacts
18 FOR EACH ROW EXECUTE FUNCTION check_artifact_rate_limit();

```

Listing 4: Rate limiting trigger function for artifact submissions

Score recalculation. After-insert, after-update, and after-delete triggers on `artifact_votes` and `analysis_votes` recompute `SUM(value)` from all votes for the affected target and write the result to the corresponding `vote_score` column. Storing the precomputed score avoids aggregation joins on every catalog query.

```

1 CREATE OR REPLACE FUNCTION sync_artifact_score_from_votes()
2 RETURNS trigger AS $$
3 DECLARE
4     target_id bigint;
5 BEGIN
6     target_id := COALESCE(NEW.artifact_id, OLD.artifact_id);
7     UPDATE artifacts
8     SET vote_score = (
9         SELECT COALESCE(SUM(value), 0)
10        FROM artifact_votes
11        WHERE artifact_id = target_id
12    )
13     WHERE id = target_id;
14     RETURN NULL;
15 END;
16 $$ LANGUAGE plpgsql SECURITY DEFINER;

```

Listing 5: Vote score synchronization trigger for artifacts

Badge refresh. Triggers on `artifacts`, `analyses`, and `collections` fire after inserts and deletes and call `refresh_badges(user_id)`. The function evaluates all six badge conditions and writes the resulting text array to `profiles.badges` in a single update.

Profile protection. A before-update trigger on `profiles` raises an exception if a non-administrator attempts to modify `is_admin`, `is_banned`, or `badges`. This guard prevents privilege escalation through direct API calls that bypass the frontend entirely.

New user initialization. An after-insert trigger on `auth.users` creates the `profiles` row, assigns the “Case Opened” badge, and resolves username collisions before the registration response reaches the client.

Timestamp maintenance. Before-update triggers on `profiles`, `artifacts`, and `analyses` set `updated_at = NOW()` automatically, keeping modification timestamps accurate without requiring application code to manage them.

14 Security Model

Security in the Internet Artifact Archive is an architectural property, not an afterthought added on top of a finished application. The following principles structure the entire design.

No client trust.

The JavaScript application can be inspected, modified, or bypassed by anyone with access to a browser's developer tools. No constraint, limit, or access rule is enforced only in JavaScript. Every rule is duplicated inside PostgreSQL. Client-side checks provide user-friendly feedback; database-level rules provide actual enforcement.

Row-level security on all tables.

Every table in the schema has `ENABLE ROW LEVEL SECURITY` and `FORCE ROW LEVEL SECURITY` applied. Every operation (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) goes through a matching policy. A table with no matching policy denies all access by default. Policies reference `auth.uid()` to identify the caller without any additional lookup or session table.

Rate limits enforced by triggers.

Insert volume limits on all content tables run as before-insert triggers inside PostgreSQL. They evaluate hourly submission counts for the calling user and reject the insert if the limit is exceeded. No amount of concurrent client requests or timing manipulation can circumvent a server-side count at write time.

Privilege escalation blocked.

The `protect_profile_update()` trigger catches any attempt to modify the `is_admin`, `is_banned`, or `badges` fields by a non-administrator and raises an exception. A carefully crafted PostgREST request body cannot elevate a regular account to administrator status.

Self-vote prevention at the policy layer.

The `INSERT` policy on `artifact_votes` evaluates `auth.uid() <> artifact_owner_id` using a cached column rather than a join, keeping the check fast and enforcement entirely server-side.

Banned user restrictions.

A helper function `is_not_banned()` is called inside the `INSERT` policy of every content table. Banned users retain read access to all public content but cannot submit artifacts, analyses, comments, votes, or collections.

Anonymous aliases validated by constraint.

The `wall_posts.alias` column carries a `CHECK` constraint using a regular expression pattern, blocking arbitrary strings from being passed as display names for anonymous posts.

15 User Interface and Experience

The interface adopts a vintage archive aesthetic: warm cream and beige backgrounds, dark brown borders, and muted status-specific accent colors (teal for active investigation, sage green for explained artifacts, rust for archived). Typography uses Inter for body text and Special Elite with Rubik Dirt for display headings, evoking printed reference catalogs and the early community sites that the platform seeks to document.

The home page features a GSAP-powered 3D card deck animation implemented in `CardSwap.jsx`. A stack of three cards representing featured artifacts cycles on a configurable interval. The front card rotates out with elastic easing while the remaining cards advance. Hovering over the deck pauses the animation. The component accepts props for card dimensions, spacing, delay, skew amount, and easing function, making it reusable for any set of content items.

Navigation is handled by the persistent `AppShell.jsx` header, which renders across all pages except the home page (where the full viewport is reserved for the landing experience). The header includes the platform logo, tab navigation, and a user menu dropdown. The dropdown shows the signed-in user's username, email, earned badge chips, and links to the dashboard, profile page, and (when applicable) admin panel. It closes automatically on route changes.

The `ErrorBoundary` component wraps the full page tree and catches any unhandled render exceptions, presenting a recovery option instead of a blank screen. All async operations that call Supabase are wrapped in `try/catch`; errors pass through `lib/errorUtils.js`, which maps every expected failure mode to a short, specific, non-technical message. A 23505 unique key violation becomes "That username is already taken." A trigger exception from a rate limit becomes "You are submitting too quickly. Please wait a moment." Users never see a raw PostgreSQL error code.

16 Conclusion

The Internet Artifact Archive delivers all ten requirements of the project brief within a domain that gives every feature a genuine, motivated purpose. Authentication protects an archive of irreplaceable cultural documentation. The CRUD model represents the full lifecycle of an investigation, from initial submission to final archival. Search and filtering allow researchers to navigate a growing corpus without scrolling through every entry. The dashboard surfaces each user's contributions at a glance. The admin panel keeps content appropriate and the community productive.

The platform extends beyond the brief in ways that reflect what a production-grade implementation requires in practice: voting with server-enforced self-vote prevention, a badge system driven entirely by database triggers, a real-time community wall supporting both authenticated and anonymous participation, peer-reviewed long-form analyses ranked by community vote, and a security model that places the database as the final authority on access control. These additions do not merely satisfy requirements; they make the platform genuinely worth using.

The architecture was chosen to be durable and extensible. Adding a new content type means adding a table and a set of RLS policies. Adding a new badge condition means updating one PostgreSQL function. Adding a new filter means adding one parameter to one fetch function. The database schema is the application's contract with its data, and the React frontend is the interface through which the community fulfills it.

Project repository: `echo_chamber/` **Database schema:** `supabase/schema.sql` **Frontend entry:** `src/App.jsx`